

# PROAUTH IQ

## AWS Deployment & Operations Guide

*ECS Fargate • AWS Copilot • RDS PostgreSQL + pgvector • S3 • ALB • SSM*

**Application: proauth-iq | Environment: test | Region: ap-south-1**

## 1. Purpose and Scope

This document is the deployment and operations reference for ProAuth IQ on AWS. It records the deployment order, architecture, commands, data migration procedure, verification steps, and redeployment process.

The deployment uses three services: a React frontend, a Node.js/Express backend, and a private Python FastAPI ML/RAG service. AWS Copilot manages the ECS Fargate environment and service infrastructure.

## 2. Deployment Architecture

Frontend and backend are exposed through a shared Application Load Balancer. The Python ML/RAG service is a private Copilot Backend Service and is reached by the backend through Service Connect (<http://ml:8002>). PostgreSQL runs on private RDS. pgvector stores policy embeddings in the same PostgreSQL database. S3 stores trained model artifacts. MongoDB Atlas stores LangGraph checkpoint state.

## 3. Why AWS Copilot CLI

Copilot provides the ECS, VPC, ALB, ECR, IAM, networking, service discovery, and CloudFormation plumbing from service manifests. This gives ProAuth IQ a production-shaped deployment without manually maintaining the complete CloudFormation/Terraform infrastructure.

## 4. AWS Components

| Component        | Technology                    | Purpose                                                    |
|------------------|-------------------------------|------------------------------------------------------------|
| Frontend         | ECS Fargate + ALB             | React/Vite web application.                                |
| Backend          | ECS Fargate + ALB             | Node/Express API and application workflow.                 |
| ML/RAG           | ECS Fargate + Service Connect | FastAPI, RAG, LangGraph agents and trained models.         |
| Database         | RDS PostgreSQL 17             | Application, policy, authorization and triage data.        |
| Vector retrieval | PostgreSQL + pgvector         | Policy embeddings and semantic retrieval.                  |
| Model storage    | S3                            | Triage and XGBoost ranker artifacts.                       |
| Secrets          | SSM / Secrets Manager         | Credentials, API keys and encryption configuration.        |
| Networking       | VPC/private subnets           | Protects RDS and the ML service from direct public access. |

## 5. Prerequisites

- AWS CLI configured
- AWS Copilot CLI installed
- Docker running
- Node.js/npm
- Python 3.11+
- MongoDB Atlas URI
- Groq API key pool
- Access to the project's real secrets

Never commit database passwords, JWT secrets, encryption keys, Groq keys, or connection strings to source control.

## 6. Step 1 — Local Docker Validation

Validate all three images before provisioning or deploying cloud services.

```
docker compose build frontend
docker compose build backend
docker compose build ml

docker compose up -d postgres
docker compose up -d ml && docker compose ps ml
docker compose up -d backend && docker compose ps backend
docker compose up -d frontend && docker compose ps frontend

docker compose down
```

## 7. Step 2 — Bootstrap Copilot

```
copilot app init proauth-iq
copilot env init --name test --region ap-south-1 --default-config
copilot env deploy --name test
```

The environment deployment creates the VPC, subnets, ECS cluster, ALB-related infrastructure and configured environment addons.

## 8. Step 3 — Enable pgvector

RDS PostgreSQL 17 is provisioned without the application extension. Create pgvector from a one-off task inside the VPC because RDS is private.

```
copilot task run \
  --app proauth-iq --env test \
  --task-group-name pgvector-init \
  --image postgres:17-alpine \
  --secrets PGPASSWORD=<DB_PASSWORD_SECRET_ARN> \
  --command 'psql -h <rds-endpoint> -U proauth -d proauth_ai -c "CREATE EXTENSION IF NOT EXISTS
vector;"' \
  --acknowledge-secrets-access --follow
```

Expected output: CREATE EXTENSION.

## 9. Step 4 — Configure Secrets

```
copilot secret init -n JWT_SECRET --values test='<random value>'
copilot secret init -n DATA_ENCRYPTION_KEY --values test='<base64 key>'
copilot secret init -n DATABASE_URL --values test='postgresql://proauth:<password>@<rds-
endpoint>:5432/proauth_ai'
copilot secret init -n MONGODB_URI --values test='<mongodb+srv URI>'
```

```
copilot secret init -n GROQ_API_KEYS
# Select test and paste the comma-separated key list when prompted
```

Generate new cryptographic values when creating a new environment:

```
openssl rand -base64 48
openssl rand -base64 32
```

The RDS password can be retrieved from its Secrets Manager secret:

```
aws secretsmanager get-secret-value --region ap-south-1 \
  --secret-id "<DB_PASSWORD_SECRET_ARN>" \
  --query "SecretString" --output text
```

## 10. Step 5 — Register Services

```
copilot svc init --name ml --svc-type "Backend Service" --dockerfile ./ProAuth_AI_ML/Dockerfile --port 8002
copilot svc init --name backend --svc-type "Load Balanced Web Service" --dockerfile ./ProAuth_AI_BackEnd/Dockerfile --port 5001
copilot svc init --name frontend --svc-type "Load Balanced Web Service" --dockerfile ./ProAuth_AI_FrontEnd/Dockerfile --port 8080
```

ML has no public ALB endpoint. Backend and frontend share the ALB.

## 11. Step 6 — Deploy ML/RAG and Upload Models

The ML container downloads its trained artifacts from S3 at startup. Therefore the first boot may restart if the expected objects are not yet present.

```
copilot svc deploy --name ml --env test

aws s3 cp ProAuth_AI_ML/policy-rag/ml/models/proauth_best_model.pkl \
  s3://<ml-model-bucket>/models/proauth_best_model.pkl

aws s3 cp ProAuth_AI_ML/priority_intelligence/models/priority_ranker.joblib \
  s3://<ml-model-bucket>/models/priority_ranker.joblib

aws s3 cp ProAuth_AI_ML/priority_intelligence/models/priority_ranker_metadata.json \
  s3://<ml-model-bucket>/models/priority_ranker_metadata.json

copilot svc deploy --name ml --env test
```

## 12. Recorded ML Deployment Recovery

The deployment record encountered a Copilot workload-addon output issue where `from_cfn: ModelsBucketName` did not resolve as expected. The recovery used the deterministic bucket name in the ML manifest and removed failed stacks before retrying.

```
aws cloudformation delete-stack --region ap-south-1 --stack-name proauth-iq-test-ml
aws cloudformation wait stack-delete-complete --region ap-south-1 --stack-name proauth-iq-test-ml
aws cloudformation delete-stack --region ap-south-1 --stack-name proauth-iq-test-ml-AddonsStack-<id>
```

Inspect CloudFormation events before deleting any stack. Do not apply this recovery procedure blindly to production resources.

## 13. Step 7 — Deploy Backend and Frontend

```
copilot svc deploy --name backend --env test
copilot svc deploy --name frontend --env test
```

After the frontend ALB URL is known, restrict backend CORS to that origin.

```
# copilot/backend/manifest.yml
variables:
  FRONTEND_URL: <frontend-ALB-origin>

copilot svc deploy --name backend --env test
```

## 14. Step 8 — Migrate Local PostgreSQL Data to RDS

The cloud database needs the application data and the `policy_chunks` vector embeddings used by RAG. The recorded deployment intentionally performs a data-only migration so historical local schema drift is not copied into the clean RDS schema.

## 14.1 Create an SSM tunnel

```
aws ecs list-tasks --cluster <cluster> \
  --service-name proauth-iq-test-backend-Service-<id> \
  --region ap-south-1 --query "taskArns[0]"

aws ecs describe-tasks --cluster <cluster> --tasks <task-id> --region ap-south-1 \
  --query "tasks[0].containers[?name=='backend'].runtimeId"

aws ssm start-session \
  --target "ecs:<cluster>_<task-id>_<runtime-id>" \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["<rds-endpoint>"],"portNumber":["5432"],"localPortNumber":["15432"]}' \
  --region ap-south-1
```

## 14.2 Dump local data

```
docker exec proauth-postgres pg_dump -U proauth -d proauth_ai \
  --data-only --disable-triggers --no-owner --no-privileges \
  -f /tmp/proauth_data.sql
```

## 14.3 Clear test RDS data

```
PGPASSWORD='<db password>' psql -h 127.0.0.1 -p 15432 -U proauth -d proauth_ai -c "
TRUNCATE TABLE users, patients, providers, insurance_plans, policies, policy_versions,
  policy_criteria, policy_chunks, authorizations, authorization_evaluations,
  triage_evaluations, documents, notifications, audit_events RESTART IDENTITY CASCADE;
"
```

## 14.4 Import

```
docker exec proauth-postgres bash -c "
  PGPASSWORD='<db password>' psql -h host.docker.internal -p 15432 -U proauth -d proauth_ai \
  -f /tmp/proauth_data.sql
"
```

The recorded import produced permission messages for DISABLE TRIGGER because the RDS master user is not a PostgreSQL superuser. Verify row counts and application behavior rather than judging success from those messages alone.

## 15. Encryption Key During Data Migration

Encrypted clinical/decision fields copied from local PostgreSQL can only be decrypted using the same DATA\_ENCRYPTION\_KEY that encrypted them. If the AWS environment uses a newly generated key, migrated ciphertext will not decrypt.

```
copilot secret init -n DATA_ENCRYPTION_KEY --values test='<local encryption key>' --overwrite

aws ecs update-service --cluster <cluster> \
  --service proauth-iq-test-backend-Service-<id> \
  --force-new-deployment --region ap-south-1
```

For production, use an explicit key-management and migration policy rather than casually reusing development keys.

## 16. pgvector / RAG Verification

- Confirm policies exist in RDS.
- Confirm policy\_chunks exist.
- Confirm embedding/vector values exist.
- Run a real authorization evaluation.
- Confirm RAG retrieves an expected policy/evidence match.

## 17. Health and End-to-End Verification

```
curl http://<ALB>/api/health
curl http://<ALB>/healthz
```

```
copilot svc logs --name ml --env test --since 10m
```

Then verify: login → authorization submission → /api/triage/evaluate → policy retrieval → decision persistence → frontend result → PEND priority ranking.

## 18. Troubleshooting

| Symptom                 | Check                                                                    |
|-------------------------|--------------------------------------------------------------------------|
| ML restarts             | Check model objects in S3, startup logs and injected secrets.            |
| HeadObject: Not Found   | Upload the exact model filenames expected by the ML endpoint.            |
| Triage returns 503      | Check ML health and backend → http://ml:8002 connectivity.               |
| Browser CORS error      | Set FRONTEND_URL to the exact frontend origin and redeploy backend.      |
| Vector extension error  | Create the vector extension on RDS.                                      |
| Decryption failure      | Verify DATA_ENCRYPTION_KEY matches the key used for migrated ciphertext. |
| RDS unreachable         | Use the SSM port-forward through a running ECS task.                     |
| RAG retrieval missing   | Verify policy_chunks and vector embeddings were migrated.                |
| CloudFormation rollback | Inspect stack events and only delete/recreate a confirmed failed stack.  |

## 19. Redeploy After a Code Change

```
copilot svc deploy --name backend --env test
copilot svc deploy --name ml --env test
copilot svc deploy --name frontend --env test
```

If a secret changed and tasks need to reload it:

```
aws ecs update-service --cluster <cluster> \
  --service <service-name> \
  --force-new-deployment --region ap-south-1
```

## 20. Live Deployment Endpoints

Recorded deployment:

- Frontend: http://proaut-Publi-o9CFLIXCkql-540122485.ap-south-1.elb.amazonaws.com
- Backend API: http://proaut-Publi-o9CFLIXCkql-540122485.ap-south-1.elb.amazonaws.com/api

These URLs are deployment-specific and may change when the environment is recreated.

## 21. Final Deployment Checklist

- ☐ Copilot app/environment available
- ☐ RDS PostgreSQL 17 healthy
- ☐ pgvector enabled
- ☐ Secrets configured

- ☐ ML service healthy and private
- ☐ Triage model in S3
- ☐ Priority XGBoost model in S3
- ☐ Backend healthy
- ☐ Frontend healthy
- ☐ CORS configured
- ☐ Application data migrated
- ☐ policy\_chunks embeddings present
- ☐ Migrated encrypted data decrypts
- ☐ Backend reaches ML via Service Connect
- ☐ Real triage evaluation succeeds
- ☐ PEND priority ranking succeeds

## 22. Quick Deployment Sequence

1. Validate Docker images
2. Initialize Copilot app/environment
3. Deploy environment infrastructure
4. Enable pgvector on RDS
5. Configure secrets
6. Register ML, backend and frontend
7. Deploy ML
8. Upload model artifacts to S3
9. Redeploy ML
10. Deploy backend and frontend
11. Configure frontend CORS origin
12. Open SSM RDS tunnel
13. Create data-only local dump
14. Clear target test data
15. Import application + policy/vector data
16. Synchronize encryption key if migrating encrypted rows
17. Force new deployment when required
18. Run health checks
19. Run real triage/RAG verification
20. Verify PEND queue ranking